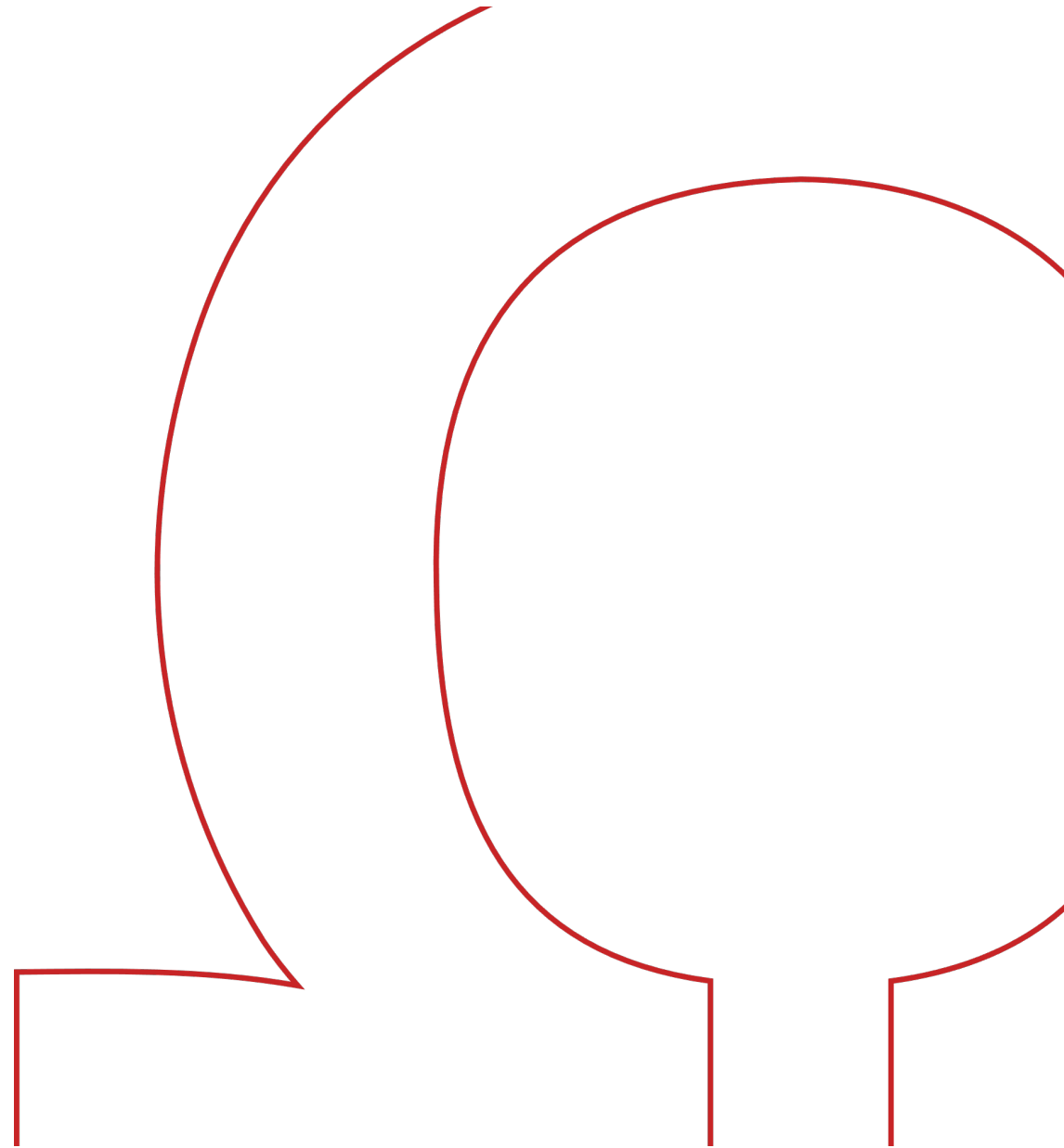


Prog III / A

Software Engineering

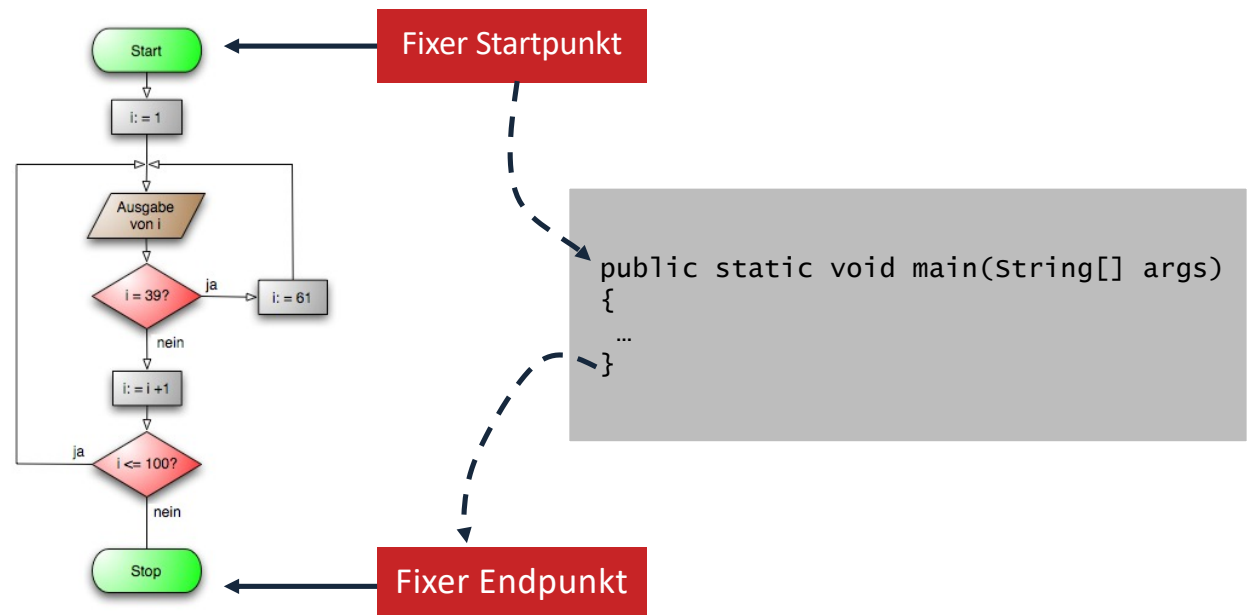


Parallelisierter Programmablauf

Parallelisierter Programmablauf

Problemstellung

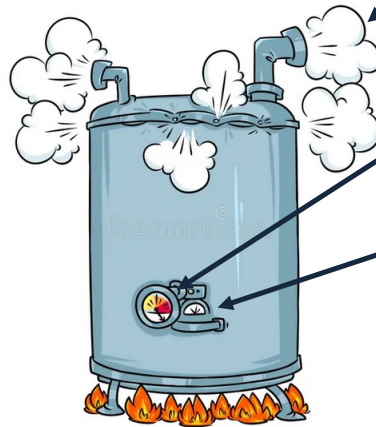
Bei einem herkömmlichen Ablauf behält das Programm vom Start bis zum Ende die Kontrolle. Allenfalls **Exceptions** können den Ablauf unterbrechen.



Parallelisierter Programmablauf

Problemstellung

Oft kann diese umfassende Kontrolle nicht gewährleistet werden. Das Programm muss vielmehr zeitnah auf Ereignisse reagieren, die ohne Einfluss des Programms jederzeit auftreten können.

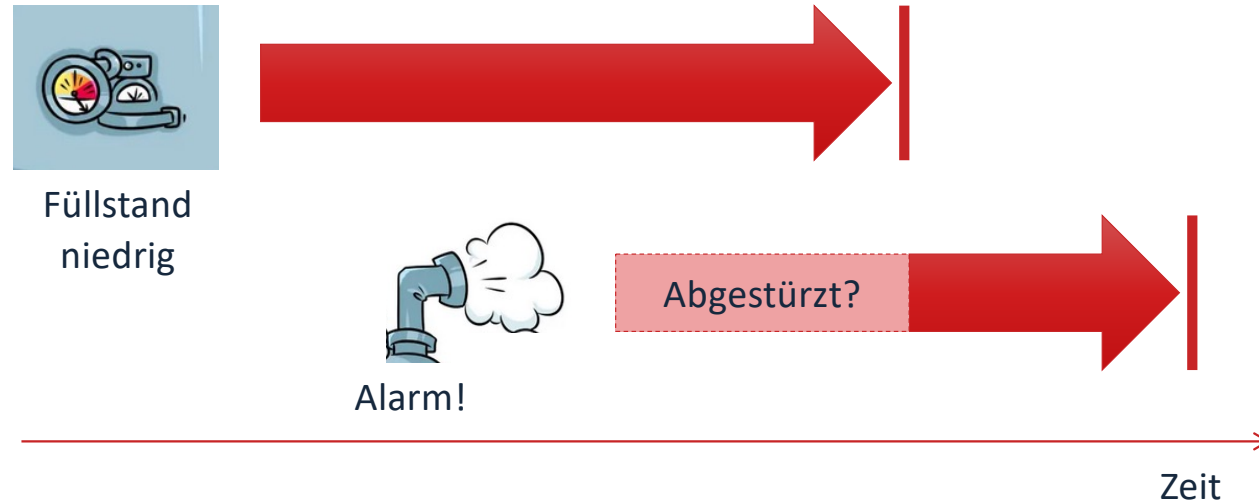


```
public void onPressureHigh()  
{  
    ...  
}  
  
public void onTemperatureHigh()  
{  
    ...  
}  
  
public void onFillingLevelLow()  
{  
    ...  
}  
...
```

Parallelisierter Programmablauf

Problemstellung

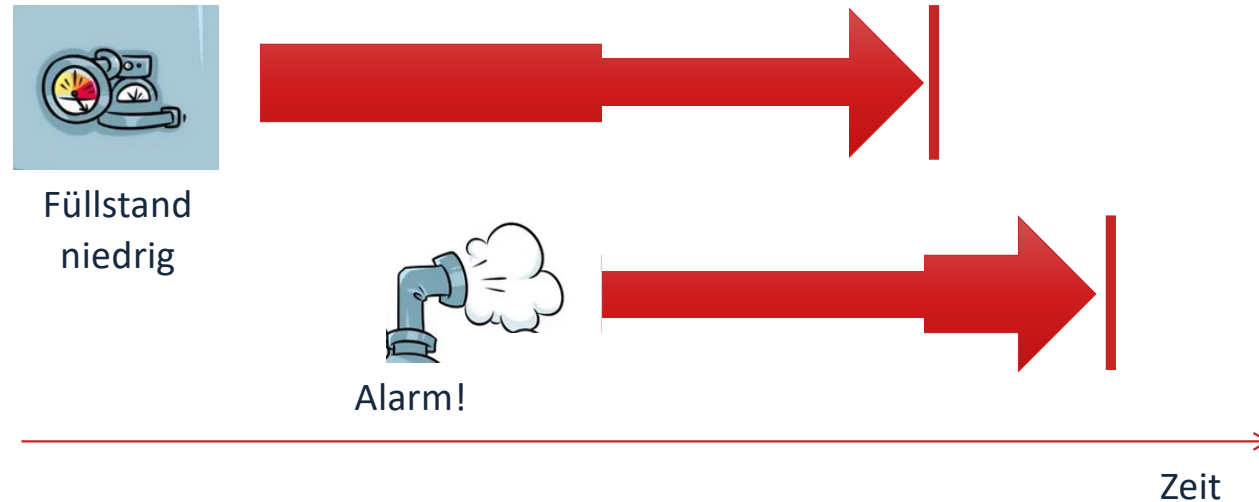
Wird ein Ereignis nach dem anderen bearbeitet, kann wertvolle Zeit für die Reaktion auf Notsituationen verstreichen und der Eindruck entstehen, dass das Programm „abgestürzt“ ist.



Parallelisierter Programmablauf

Besser: Quasi-Parallele Abarbeitung

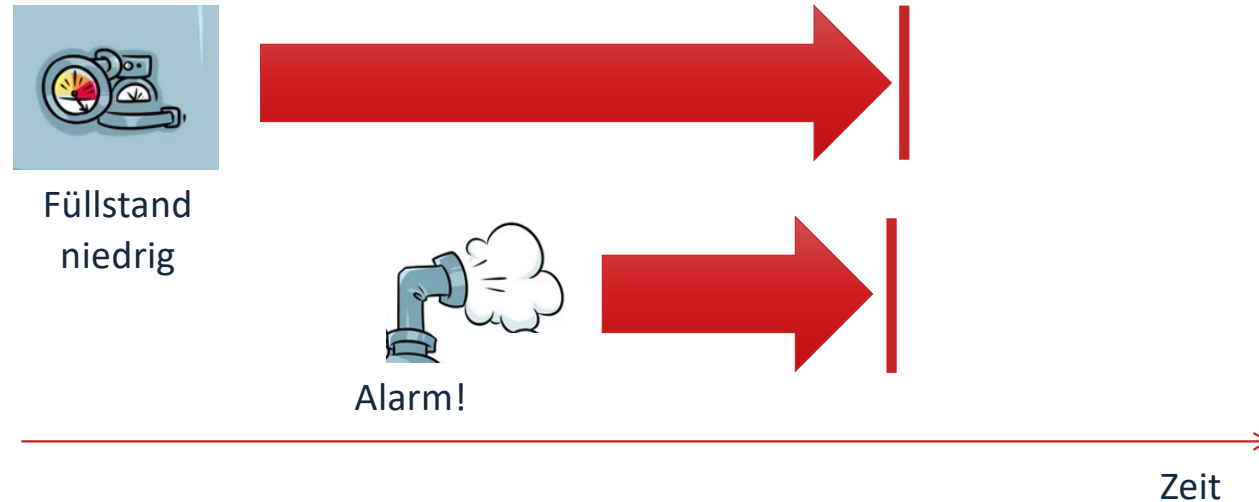
Man wartet nicht, bis die Behandlung vom ersten Ereignis abgeschlossen ist, sondern zweigt einen Teil der Rechenleistung bereits zur Behandlung vom zweiten Ereignis ab (d.h. die Rechenleistung wird geteilt).



Parallelisierter Programmablauf

Noch besser: Echte parallele Abarbeitung

Stehen mehrere Prozessoren bzw. Prozessorkerne zur Verfügung, dann kann die Bearbeitung tatsächlich parallel erfolgen.



Parallelisierter Programmablauf

Konzeptioneller Ansatz

Wie können Programmteile programmiert werden,
die gleichzeitig zur Ausführung kommen?

→ Threads

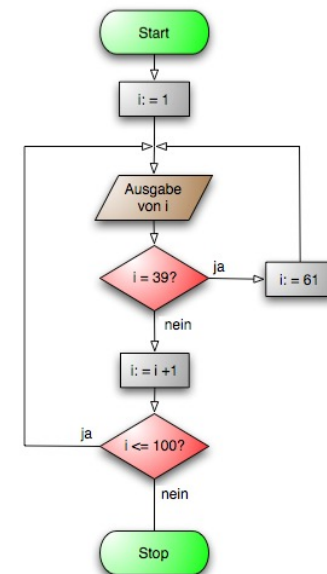


Threads

Threads

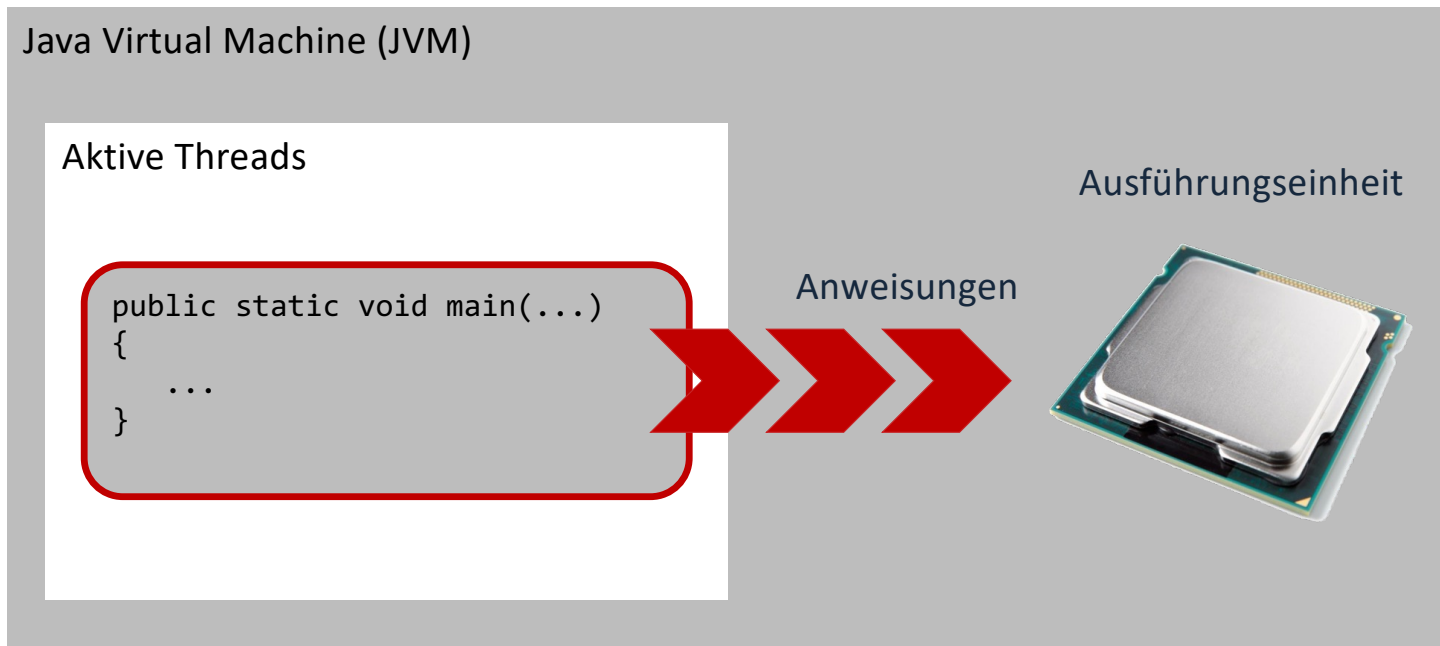
Single-Thread Programme

- Unmittelbar mit dem Programmstart wird von der Java-Laufzeitumgebung ein sog. **Thread** angelegt und gestartet.
- In diesem **Thread** beginnt die Ausführung der main-Methode.
- Innerhalb des **Threads** wird mit einem Befehlszähler festgehalten, welche Anweisung gerade ausgeführt wird.
- Nach der Ausführung der Anweisung wird der Befehlszähler weitergeschaltet bzw. bei Methodenaufrufen oder Schleifen auf die nächste Anweisung gesetzt.
- Der **Thread** beendet sich („terminiert“), wenn der Befehlszähler auf keine weitere Anweisung mehr verweist, d.h. main abgearbeitet ist.



Threads

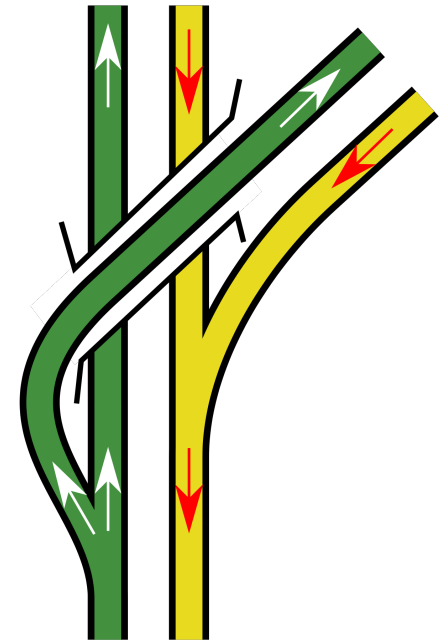
Single-Thread Programme



Threads

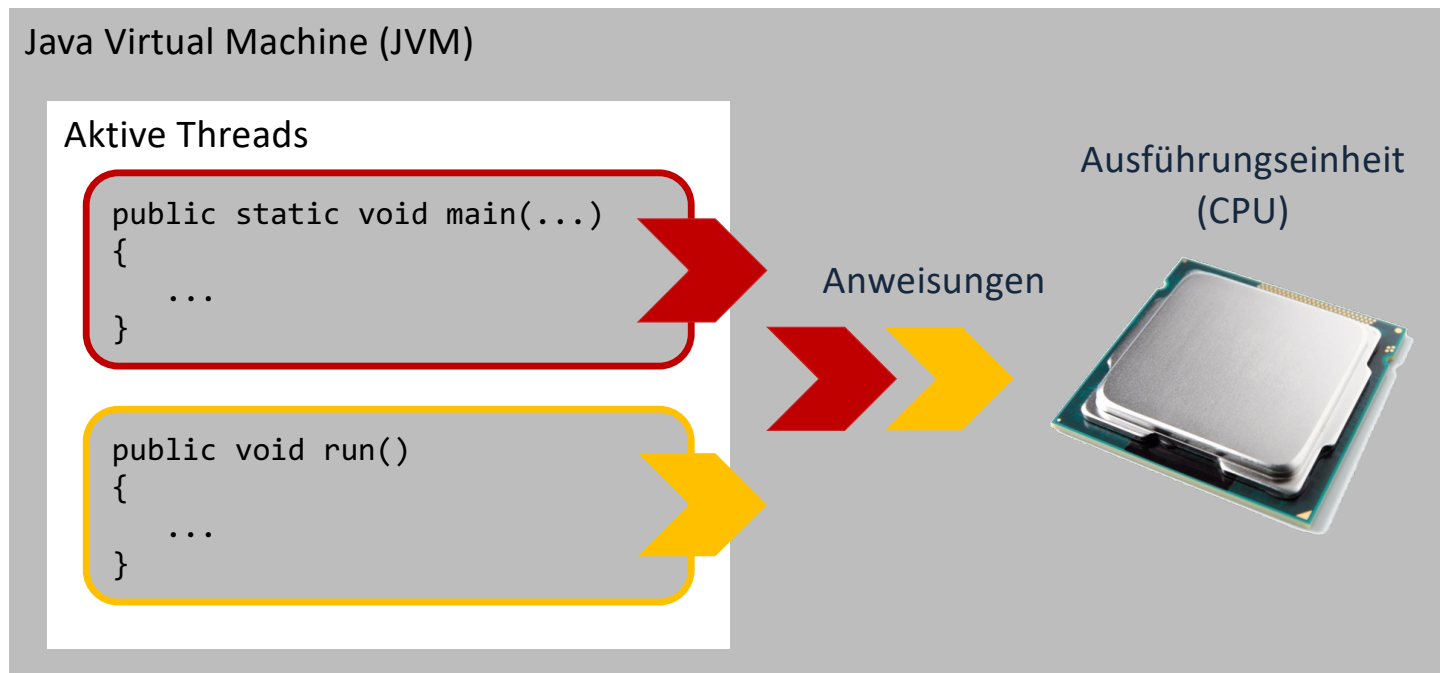
Multi-Thread Programme

- Während des Programmablaufs können jederzeit weitere **Threads** erzeugt werden.
- Diese besitzen einen eigenen Befehlszähler und einen eigenen **Stack**, der **Heap** und **globale Variablen** werden aber gemeinsam genutzt.
- Der von einem neu erzeugten **Thread** auszuführende Code kann festgelegt werden, d.h. der **Thread** startet nicht die `main`-Methode.
- Die Java Virtual Machine (JVM) sorgt dafür, dass alle aktiven **Threads** abwechselnd (bzw. bei mehreren Prozessorkernen gleichzeitig) zum Zuge kommen und Anweisungen an die Ausführungseinheit weiterreichen können.



Threads

Multi-Thread Programme



ohm

Threads

Die JVM enthält einen Scheduler, der die Rechenzeit auf die aktiven Threads verteilt (Multitasking).



Threadprogrammierung in Java

Threadprogrammierung in Java

Interface Runnable

Der parallel auszuführende Code wird in einer eigenen Klasse implementiert, die das Interface Runnable erfüllen muss. Das Interface erzwingt eine Methode run, die den Code enthält.

```
public class Task1 implements Runnable
{
    public void run()
    {
        long start = System.currentTimeMillis();
        while (System.currentTimeMillis() < start + 2000)
            ;
        System.out.println("Task 1 fertig");
    }
}
```


Threadprogrammierung in Java

Klasse Thread

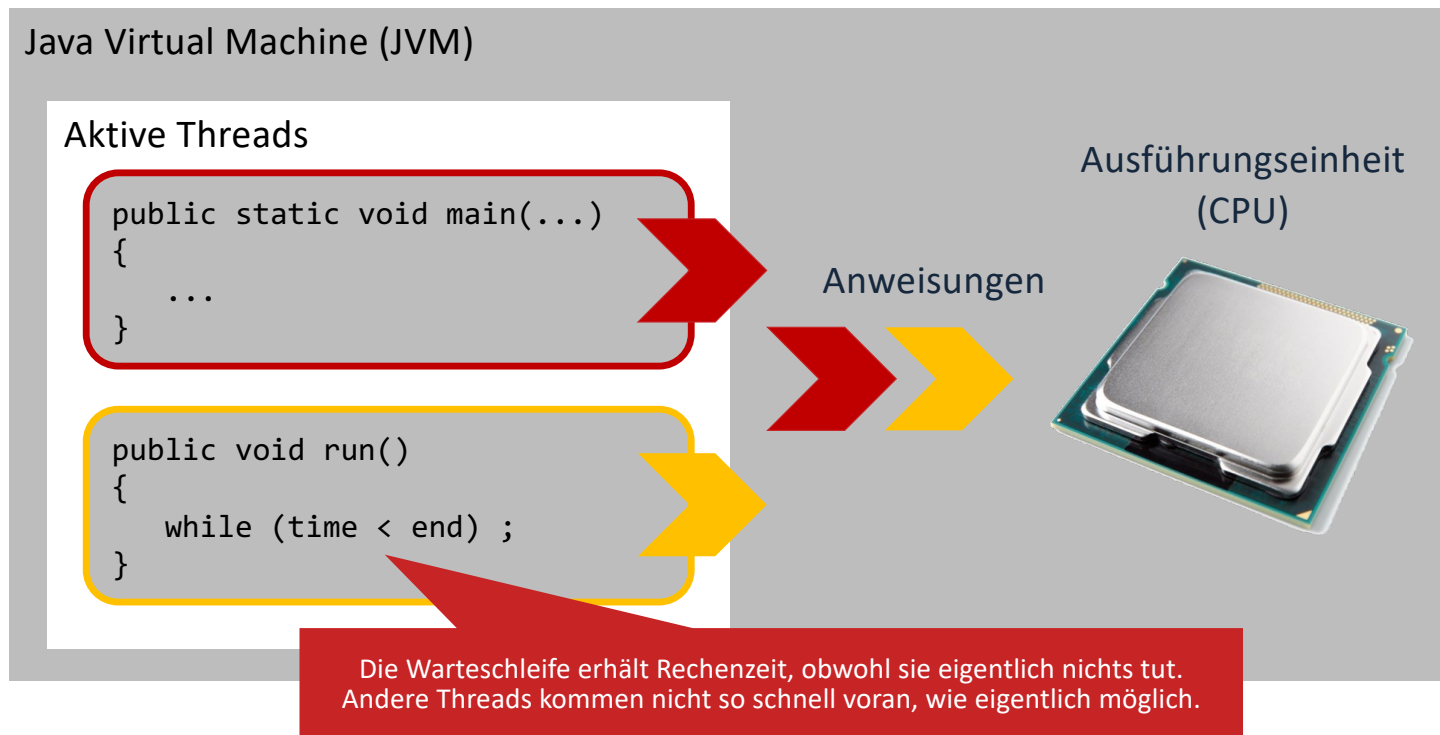
Zur Anlage eines weiteren **Threads** kann die Bibliotheksklasse Thread verwendet werden. Bei der Erzeugung kann eine Runnable-Instanz im Konstruktor an das neue Thread-Objekt übergeben werden.

Die Ausführung des neuen **Threads** beginnt, sobald die Methode start aufgerufen wird.

```
public static void main(String[] args)
{
    Runnable parallelerCode = new Task1();
    Thread thread = new Thread(parallelerCode);
    thread.start();
    System.out.println("main Ende!");
}
```

Threadprogrammierung in Java

Busy Wait



Threadprogrammierung in Java

Busy Wait

Bei einem **Busy Wait** wird eine Pause eines Threads dadurch realisiert, dass wiederholt überprüft wird, ob die Pause beendet ist.

Dazu muss der Thread immer wieder Rechenzeit erhalten, obwohl er eigentlich keine benötigt.

Die Rechenzeit fehlt den anderen parallelen Threads.



Threadprogrammierung in Java

Busy Wait – etwas besser

Mit `Thread.yield` kann dem Scheduler mitgeteilt werden, dass der aktuelle Thread gerade keine Rechenzeit benötigt und ein anderer Thread bearbeitet werden kann.

```
public class Task1 implements Runnable {  
    public void run() {  
        long start = System.currentTimeMillis();  
        while (System.currentTimeMillis() < start + 2000)  
            Thread.yield();  
        System.out.println("Task 1 fertig");  
    }  
}
```

Threadprogrammierung in Java

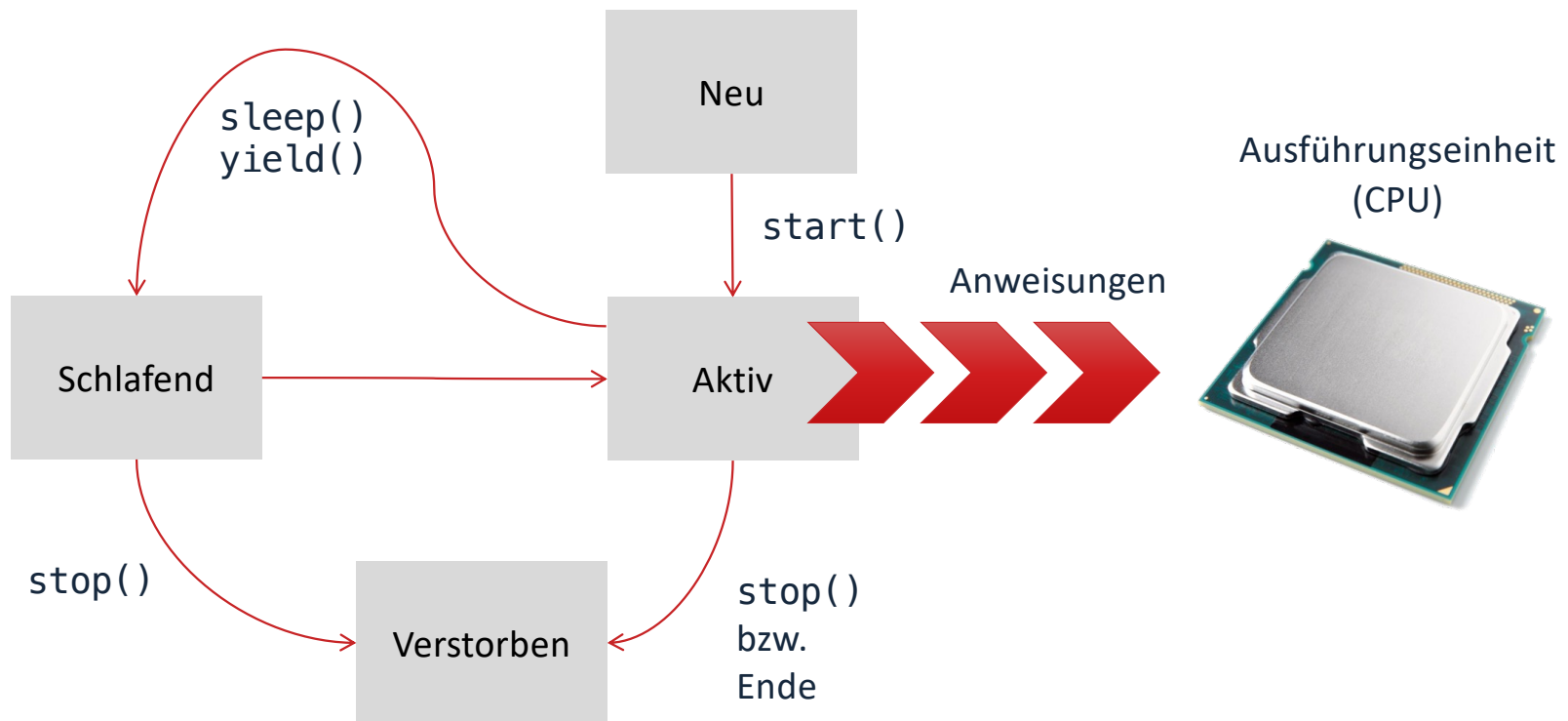
Busy Wait – Optimale Lösung

Mittels `Thread.sleep` wird der ausführende Thread in einen Schlafmodus versetzt, aus dem er erst nach Verstreichen der angegebenen Millisekunden wieder geweckt wird.

```
public class Task2 implements Runnable
{
    public void run()
    {
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("Unterbrochen!");
        }
        System.out.println("Task 2 fertig");
    }
}
```

Threadprogrammierung in Java

Threadzustände



Threadsynchronisation

Threadsynchronisation

Problemstellung

- Werden in einem Programm mehrere Threads gestartet, dann läuft das Programm, solange es noch mindestens einen schlafenden und/oder aktiven Thread gibt.
- Auch nach dem Ende der `main`-Methode kann das Programm somit noch weiterlaufen.
- Oft ist es aber notwendig, dass Threads aufeinander warten, weil z.B. Ergebnisse weitergereicht werden müssen.
- Die Koordinierung des zeitlichen Ablaufs mehrerer Threads nennt man ***Threadsynchronisation***.



Threadsynchronisation

Join

Im einfachsten Fall möchte ein Thread auf das Ende eines anderen Threads warten. Dazu kann er sich mittels `join()` schlafen, bis der andere Prozess endet.

```
public static void main(String[] args)
{
    Runnable parallelerCode = new Task2();
    Thread thread = new Thread(parallelerCode);
    thread.start();
    try {
        thread.join();
    } catch (InterruptedException e) {
        System.out.println("Unterbrochen!");
    }
    System.out.println("main Ende!");
}
```

Threadsynchronisation

Race Conditions

Alle Threads eines Programms teilen sich einen gemeinsamen Adressraum.

Alle globalen Objekte/Instanzen sind von allen Threads aus zugänglich.

```
public static void main(String[] args) {  
    (new Thread(new RaceTask())).start();  
    (new Thread(new RaceTask())).start();  
}
```

```
public class RaceTask implements Runnable {  
  
    static int counter = 0;  
  
    public void run() {  
        while (counter < 100) {  
  
            // 1. Zugriff auf globale Variable  
            int localCounter = counter;  
  
            try {  
                int sleepTime = (int) (Math.random() * 100);  
                Thread.sleep(sleepTime);  
            } catch (InterruptedException e) {  
                System.out.println("Unterbrochen!");  
            }  
  
            // 2. Zugriff auf globale Variable  
            int diff = counter - localCounter;  
  
            // Ups?  
            if (diff != 0)  
                System.out.println("Wait what?!");  
            else  
                counter++;  
        }  
    }  
}
```

Threadsynchronisation

Locking

Mit Hilfe von `synchronized` kann ein **kritischer Abschnitt** definiert werden, der immer nur von einem Thread betreten werden darf.

Notwendig ist hierfür eine von allen betroffenen Threads zugängliche Instanz eines Referenztyps (hier: `lock`). Die JVM führt eine Warteschlange aller Threads, die den **kritischen Abschnitt** betreten wollen, lässt immer nur einen in den Abschnitt und legt alle anderen Threads schlafen, bis der Abschnitt verlassen wird.

Nachteil:

Ggf. starke Beeinträchtigung der Parallelität,
Gefahr von Deadlocks

```
public class SyncRaceTask implements Runnable {

    static int counter = 0;
    static final Object lock = new Object();

    public void run() {
        while (counter < 100) {

            synchronized (lock) {
                // 1. Zugriff auf globale Variable
                int localCounter = counter;

                try {
                    int sleepTime = (int) (Math.random() * 100);
                    Thread.sleep(sleepTime);
                } catch (InterruptedException e) {
                    System.out.println("Unterbrochen!");
                }

                // 2. Zugriff auf globale Variable
                int diff = counter - localCounter;

                // Ups?
                if (diff != 0)
                    System.out.println("Wait what?!" );
                else
                    counter++;
            }
        }
    }
}
```

Threadsynchronisation

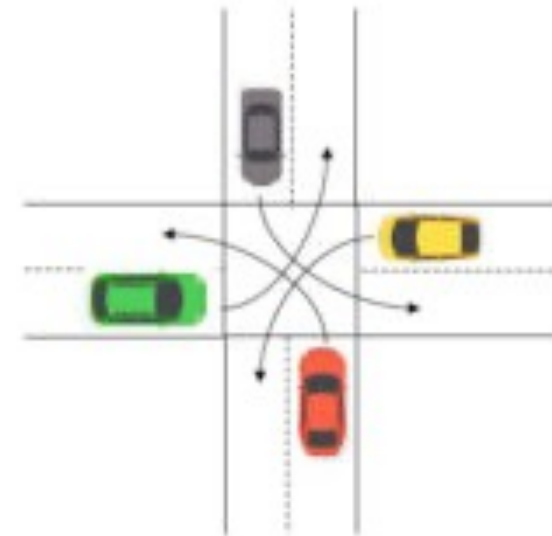
Deadlock

Ein Deadlock (auch Verklemmung) entsteht, wenn Threads mehrere kritische Abschnitte betreten wollen und sich wechselseitig daran hindern.

Beispiel:

Vier Autos fahren auf eine Kreuzung und konkurrieren um die Weiterfahrt. Es gilt die Rechts-vor-Links-Regel.

Ein Deadlock ist entstanden, da ohne weitere Verständigung kein Auto weiterfahren kann. Aufgelöst werden kann diese Verklemmung durch eine (externe) Entscheidung, welches Auto als erstes weiterfahren darf.

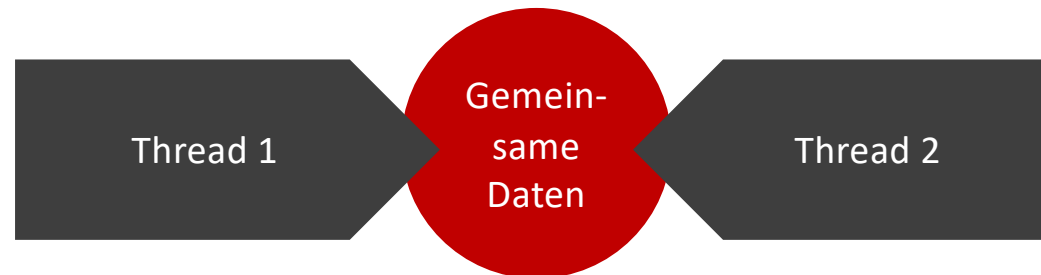


Thread Safety

Thread Safety

Begriff

Thread Safety bezieht sich auf die Fähigkeit eines Moduls, einer Klasse oder einer Funktion, sicher von mehreren Threads gleichzeitig verwendet zu werden, ohne dass es zu unerwartetem Verhalten kommt.



Erreicht wird das mittels **Threadsynchrisation** z.B. durch **kritische Abschnitte**, um **Race Conditions** zu vermeiden. Gleichzeitig ist auch sicherzustellen, dass keine **Deadlocks** auftreten können.

Thread Safety

Java Standardbibliothek

Die Klassen der Java Standardbibliothek sind nicht zwingend *thread safe*.

Was kann im folgenden Beispiel schief gehen, wenn beide Tasks gestartet werden und `list` eine `ArrayList` enthält?

Producer-Task

```
public static class Producer
    implements Runnable
{
    public void run() {
        for (int i=0; i<100; i++)
        {
            list.add(i);
            Thread.yield();
        }
    }
}
```

Consumer-Task

```
public static class Consumer
    implements Runnable
{
    public void run() {
        for (int i=0; i<100; i++)
        {
            while (list.isEmpty())
                Thread.yield();
            list.remove(0);
        }
    }
}
```

Thread Safety

Java Standardbibliothek

Die Klassen der Java Standardbibliothek sind nicht zwingend *thread safe*.

Wenn es sich bei der Liste um z.B. eine `ArrayList` handelt, kann es zu einer *Race Condition* kommen.



Thread Safety

Java Standardbibliothek

Verwendet man stattdessen den **Decorator** `Collections.synchronizedList`, so erhält man ein neues Objekt, das ebenfalls das `List`-Interface erfüllt, die ursprüngliche Liste zur Datenhaltung verwendet, aber alle Operationen jeweils in einem kritischen Abschnitt ausführt. Dieses neue Listenobjekt ist dann **thread safe**.

```
// Nicht Thread-sicher!  
static List<Integer> list = new ArrayList<Integer>();  
  
// Thread-sicher!  
static List<Integer> syncList = Collections.synchronizedList(list);
```

Vielen Dank für die Aufmerksamkeit!

Fragen?